HACETTEPE UNIVERSITY

ELECTRICAL AND ELECTRONICS ENGINEERING

ELE432 PRELIMINARY WORK – 3

EYYUP CAN ARSLAN

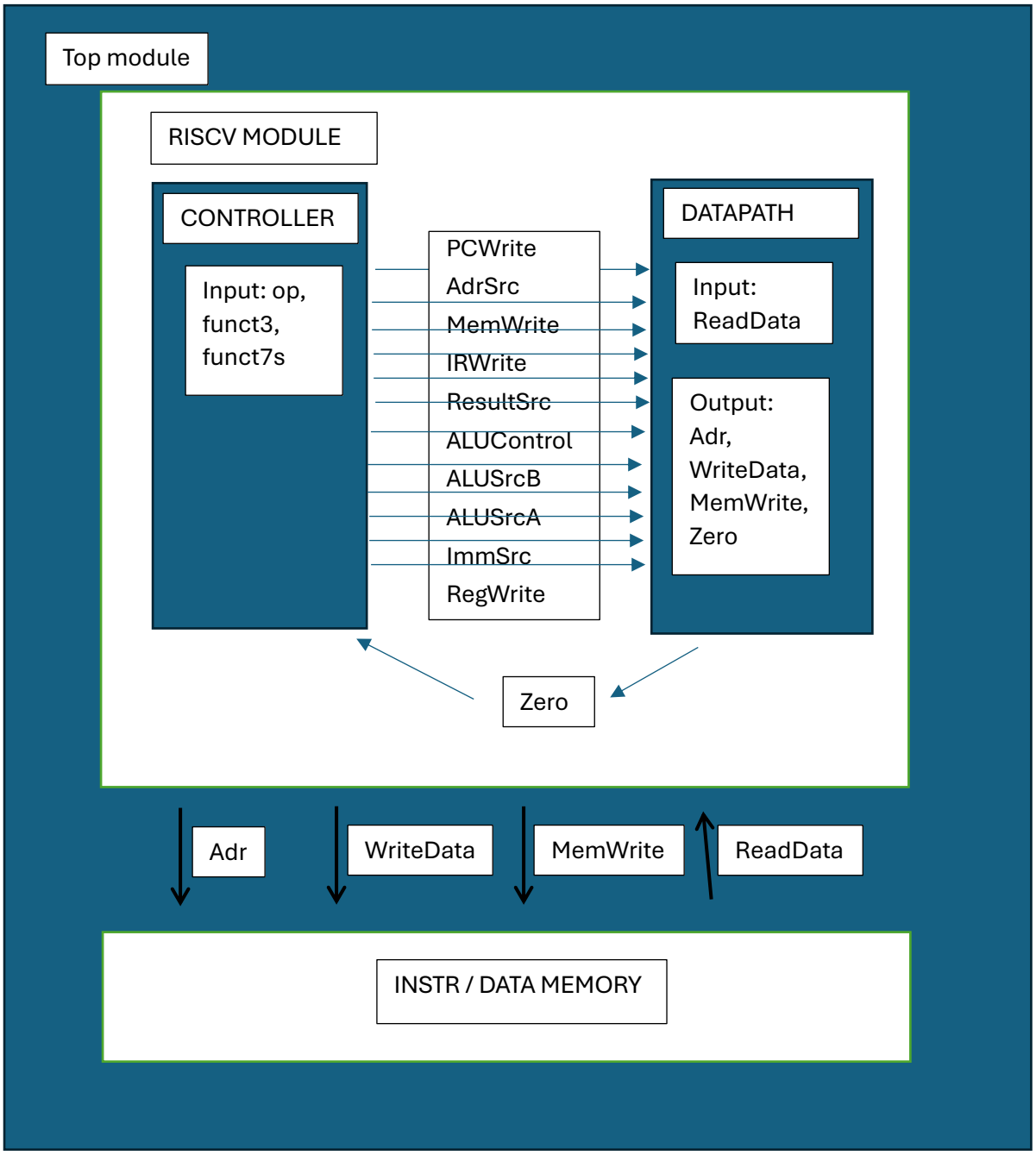
21946845

https://github.com/EyyupArslan/RISC-V-Multicycle-Processor.git

1.Please indicate how many hours you spent on this lab. This will be helpful for calibrating the workload for next time the course is taught.

5-6 hours

2. Diagram showing your memory, riscv, datapath, and controller block hierarchy and names of all signals between them



3. Hierarchical SystemVerilog for your top-level processor module (and submodules) matching the declaration given above.

Controller

```
// controller.sv
module controller(input  logic [6:0] op,
                  input  logic [2:0] funct3,
                  input  logic      funct7b5,
                  input  logic      Zero,
                  input  logic      clk, reset,
                  output logic [1:0] ImmSrc,
                  output logic [1:0] ALUSrcA, ALUSrcB,
                  output logic [1:0] ResultSrc,
                  output logic [2:0] ALUControl,
                  output logic      IRWrite, PCWrite, RegWrite, MemWrite, AdrSrc);

  typedef enum logic [3:0] {FETCH, DECODE, MEMADR, MEMREAD, MEMWB, MEMWRITE, EXECUTR, ALUWB, EXECUTI, JAL, BEQ}
statetype;
  statetype state, nextstate;
  logic [1:0] ALUOp;
  logic Branch, PCUpdate;

  always_ff @(posedge clk or posedge reset)
    if (reset) state <= FETCH;
    else      state <= nextstate;

  always_comb
    case (state)
      FETCH:   nextstate = DECODE;
      DECODE:  case(op)
                 7'b0000011: nextstate = MEMADR;
                 7'b0100011: nextstate = MEMADR;
                 7'b0110011: nextstate = EXECUTR;
                 7'b0010011: nextstate = EXECUTI;
                 7'b1101111: nextstate = JAL;
                 7'b1100011: nextstate = BEQ;
                 default:    nextstate = FETCH;
               endcase
      MEMADR:  case(op)
                 7'b0000011: nextstate = MEMREAD;
                 7'b0100011: nextstate = MEMWRITE;
                 default:    nextstate = FETCH;
               endcase
      MEMREAD:  nextstate = MEMWB;
      EXECUTR:  nextstate = ALUWB;
      EXECUTI:  nextstate = ALUWB;
      default:  nextstate = FETCH;
    endcase

  always_comb begin
    {PCUpdate, AdrSrc, MemWrite, IRWrite, RegWrite, Branch} = 6'b0;
    {ALUSrcA, ALUSrcB, ResultSrc} = 6'b0;
    ALUOp = 2'b00;
    case (state)
      FETCH:   begin AdrSrc = 0; IRWrite = 1; ALUSrcA = 2'b00; ALUSrcB = 2'b10; ALUOp = 2'b00; ResultSrc = 2'b10; PCUpdate = 1; end
      DECODE:  begin ALUSrcA = 2'b01; ALUSrcB = 2'b01; ALUOp = 2'b00; end
      MEMADR:  begin ALUSrcA = 2'b10; ALUSrcB = 2'b01; ALUOp = 2'b00; end
      MEMREAD: begin AdrSrc = 1; end
      MEMWB:   begin ResultSrc = 2'b01; RegWrite = 1; end
      MEMWRITE:begin AdrSrc = 1; MemWrite = 1; end
      EXECUTR: begin ALUSrcA = 2'b10; ALUSrcB = 2'b00; ALUOp = 2'b10; end
      ALUWB:   begin ResultSrc = 2'b00; RegWrite = 1; end
      EXECUTI: begin ALUSrcA = 2'b10; ALUSrcB = 2'b01; ALUOp = 2'b10; end
      BEQ:     begin ALUSrcA = 2'b10; ALUSrcB = 2'b00; ALUOp = 2'b01; ResultSrc = 2'b00; Branch = 1; end
    endcase
  end
  assign PCWrite = (Branch & Zero) | PCUpdate;
  // ImmSrc ve ALU Decoder kısımlarını da buraya ekleyebilirsin.
endmodule
```

Datapath

```
// datapath.sv
module datapath(input  logic clk, reset,
            input  logic [1:0] ResultSrc, ALUSrcA, ALUSrcB, ImmSrc,
            input  logic [2:0] ALUControl,
            input  logic      IRWrite, PCWrite, RegWrite, AdrSrc,
            output logic [6:0] op,
            output logic [2:0] funct3,
            output logic      funct7b5, Zero,
            output logic [31:0] Adr, WriteData,
            input  logic [31:0] ReadData);

  logic [31:0] PC, Instr, Data, A, RD1, RD2, SrcA, SrcB, ALUResult, ALUOut, Result, ImmExt;

  always_ff @(posedge clk) begin
    if (PCWrite) PC <= Result;
    if (IRWrite) Instr <= ReadData;
    Data <= ReadData;
    A <= RD1;
    WriteData <= RD2;
    ALUOut <= ALUResult;
  end

  assign Adr = AdrSrc ? ALUOut : PC;
  // Mux ve Modül bağlantıları (alu, regfile, extend) burada yer almalı.
endmodule
```

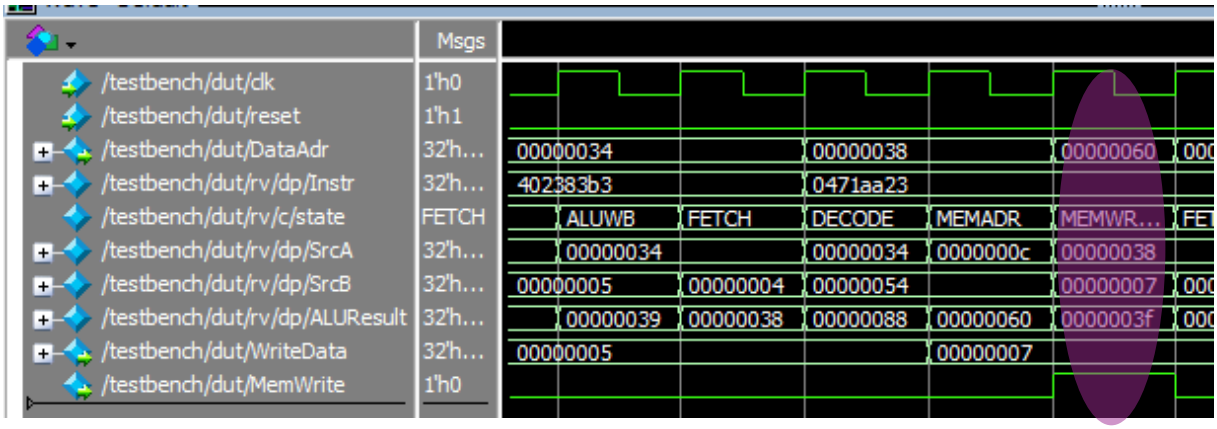
RISC-V Top Level

```
// riscv.sv
module riscv(input  logic clk, reset,
          output logic [31:0] Adr, WriteData,
          output logic MemWrite,
          input  logic [31:0] ReadData);
  // Sinyal tanımlamaları ve modül çağırmaları (instantiation)
endmodule
```

4. Table 1 showing key signals for at least the first three instructions.

Step	PC	Instr	State	Result	Result Notes
3	00	00500113	S0: Fetch	4	PC+4
4	04	""	S1: Decode	X	OldPC+Immediate
5	04	""	S8: ExecuteI	X	ALUResult = x0 (0) + 5 = 5
6	04	""	S7: ALUWB	5	Result = ALUOUT
7	04	""	S0: Fetch	8	PC+4
8	08	00c00193	S1: Decode	X	OldPC+Immediate
9	08	00c00193	S8: ExecuteI	C	ALUResult = x0 + 12 = 12 (Hex: C)
10	08	00c00193	S7: ALUWB	C	12 was written to the x3 register.
11	08	ff718393	S0: Fetch	0C	PC+4
12	0C	ff718393	S1: Decode	X	OldPcImmediate
13	0C	ff718393	S8: ExecuteI	3	ALUResult = x3 (12) + (-9) = 3
14	0C	ff718393	S7: ALUWB	3	3 was written to the x7 register.
15	0C	0023e233	S0: Fetch	10	4. Transfer of command
16					
17					
18					
19					
20					
21					
22					
23					
24					
25					
26					
27					
28					
29					
30					
31					
32					
33					
34					
35					
36					
37					
38					
39					
40					
41					
42					
43					

5. Simulation waveforms (in the order listed above: clk, reset, PC, Instr, state, SrcA, SrcB, ALUResult, Adr, WriteData, and MemWrite – all displayed in hexadecimal for ease of reading). Does your system pass your testbench? Circle or highlight the waves showing that the correct value is written to the correct address, and make sure it is legible.



In this lab, I successfully designed and simulated a **Multicycle RISC-V Processor** using SystemVerilog. By implementing a **Finite State Machine (FSM)**, I managed to break down instruction execution into multiple steps like **Fetch, Decode, and Execute**, which allows for better hardware reuse compared to single-cycle designs.

The simulation in Questasim using the riscvtest.txt file confirmed that the processor works perfectly. As seen in the waveforms, all control signals transition correctly between states, and the ALU performs the expected calculations. The system passed the testbench by successfully writing the correct success code to memory, proving that both the controller and the datapath are fully functional and properly integrated.

https://github.com/EyyupArslan/RISC-V-Multicycle-Processor.git